Design Document

USBMalwareActivityMonitoring System

PBL Phase2:DesignandPrototype DSA and Language: C++ (OOP

1. Purpose

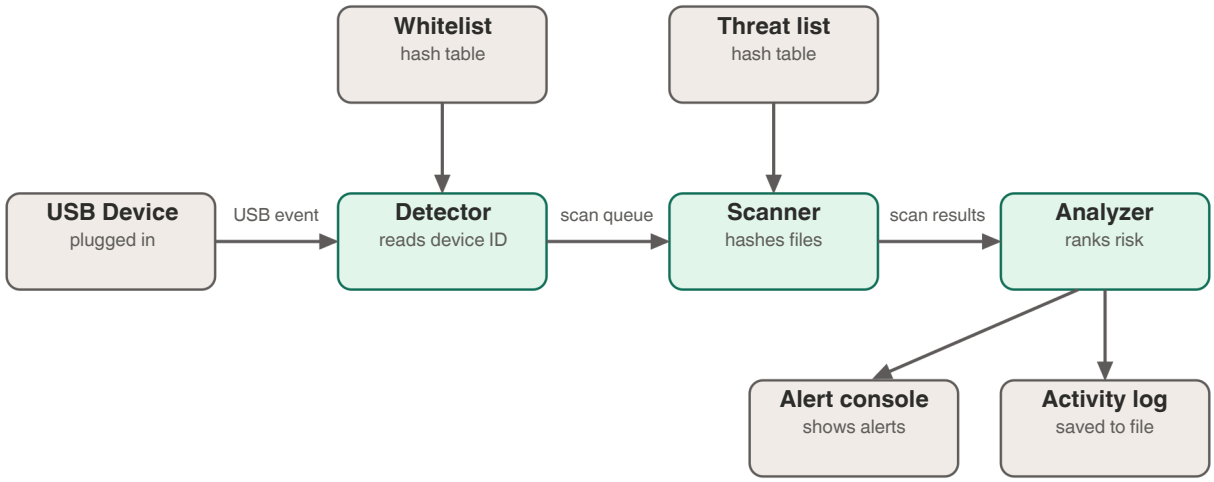
Thisdocument explains how the three core parts of the system, the **Detector**, the **Scanner** and the **Analyzer**, are designed and how they connect. The system watches for USB devices being plugged in, scans the files on them, and warns the user when something looks malicious. Each part is built and tested on its own first, using simulated USB events and safe sample files, so no real malware is ever used.

2. Objectives

- Detect when a USB device is inserted and read its basic details (vendor ID, product ID, serial number).
- Scan the files on the device and compare their fingerprints (hashes) with a list of known threats.
- Rank suspicious findings by severity so the most serious ones are handled first.
- Show alerts to the user and keep a permanent activity log.
- Apply C++ OOP and data structures and algorithms (DSA) concepts in a real security problem.

3. System architecture

A USB event enters the Detector, which hands the device to the Scanner through a queue. The Scanner passes its results to the Analyzer, which produces alerts and log entries. Two lookup tables, a whitelist and a threat list, support the Detector and the Scanner.



4. Module design

Module	Responsibility	Input	Output	DSA used
Detector	Notices a new USB device, reads its vendor ID, product ID and serial number, and checks it against the whitelist. Trusted devices are skipped. Unknown devices are sent for scanning.	USB insertion event	A USBDevice object placed in the scan queue	Hash table (whitelist), queue
Scanner	Takes the next device from the queue, walks through its folders, computes a hash of each file and looks it up in the threat list.	USBDevice from the queue	Scan results: file path, hash, match found or not	Hash table (threat list), stack for folder traversal, hashing
Analyzer	Gives each finding a risk level (low, medium or high), puts alerts in order of severity and sends them to the logger.	Scan results	Ranked alerts and log entries	Priority queue (heap), sorting

5. How the modules will be connected

1. AUSBdevice isplugged in and theoperating system raises an event. In Phase 3 this event will be simulated.
2. The **Detector** will read the device details, checks the whitelist and pushes unknown devices into the scan queue.
3. The **Scanner** will remove the next device from the queue, lists its files, hashes each one and will compare the hash with the threat list.
4. The **Analyzer** will receive the results, assigns a risk level and stores the alerts in a priority queue so high-risk ones come first.
5. The **Logger** will show each alert on the alert console and will write every event to the activity log file.

6. Class design

Each module is a C++class. A **Monitor** class owns the Detector, Scanner and Analyzer and runs them in order, so each class stays small and can be tested separately. The Detector creates USBDevice objects, the Scanner reads them, and the Analyzer sends alerts to the Logger.

